



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT III – CLASSIFICATION

K-Nearest Neighbor Algorithm and Design Considerations

1. Introduction

1.1 Definition

The K-Nearest Neighbor (kNN) algorithm is a non-parametric, instance-based (lazy learning) classification and regression method. It classifies a new data point based on the majority class among its k closest neighbors in the training data, measured by a chosen distance metric.

Definition:

kNN: classifies a query point by majority vote among its k nearest training points (by distance).

Lazy learning: kNN performs no explicit training phase — it simply stores the training data and defers all computation to prediction time.

Non-parametric: kNN makes no assumption about the underlying data distribution (unlike Naive Bayes' distributional assumptions).

1.2 Importance in Machine Learning

kNN is one of the simplest and most intuitive ML algorithms, often used as a baseline and in domains where decision boundaries are highly irregular and not well captured by linear or parametric models. It underpins many real systems: recommendation engines, anomaly detection, image recognition, and is the conceptual basis for more advanced techniques (e.g., kernel density estimation, k-means clustering).

1.3 Role in the Classification Pipeline

kNN is typically applied after feature scaling and dimensionality reduction (PCA), since it is acutely sensitive to feature scale and the curse of dimensionality. It serves as both a standalone classifier and a useful sanity-check/baseline model for benchmarking more complex algorithms like SVM and ensemble methods.

2. Learning Outcomes

Understand

- The lazy-learning mechanism of kNN: no training phase, all computation at prediction time
- The bias-variance trade-off as a function of k
- The curse of dimensionality and its impact on distance-based classification

Apply

- Implement kNN classification using scikit-learn with proper feature scaling
- Select an appropriate distance metric for numeric, categorical, and mixed-type data

- Tune k using cross-validation and interpret the resulting error curve

Analyze

- Diagnose overfitting (small k) vs underfitting (large k) from training/validation error curves
- Evaluate the computational trade-offs of different nearest-neighbor search structures
- Design a kNN pipeline robust to class imbalance and high dimensionality

3. Detailed Technical Concepts

3.1 The kNN Algorithm

Given a training set $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ and a query point x_q , kNN classification proceeds as follows:

- Step 1: Compute the distance $d(x_q, x_i)$ between the query point and every training point x_i .
- Step 2: Select the k training points with smallest distance to x_q (the " k nearest neighbours").
- Step 3: For classification, assign x_q the majority class label among the k neighbours (plurality vote). For regression, assign the mean (or weighted mean) of the k neighbours' target values.

Because there is no explicit model-fitting step (just storing the training data), kNN is called a "lazy learner" — in contrast to "eager learners" like Naive Bayes or Decision Trees that build an explicit model during training.

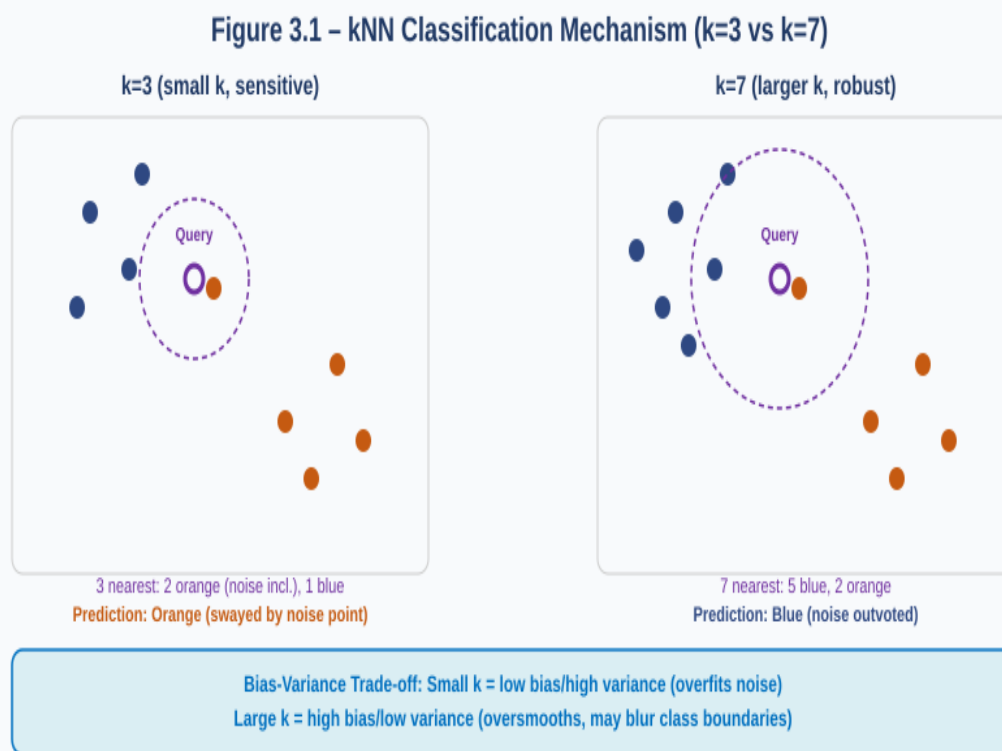


Figure 3.1 – kNN Classification Mechanism: Effect of k on Noise Sensitivity

3.2 Choosing the Value of k

The value of k controls the bias-variance trade-off central to kNN performance:

- Small k (e.g., $k=1$): Low bias, high variance. The decision boundary tightly follows individual training points, including noise/outliers — prone to overfitting.

- Large k: High bias, low variance. The decision boundary becomes smoother, but may blur genuine class boundaries — prone to underfitting, and at the extreme (k=n) always predicts the majority class.
- Odd k for binary classification: avoids tied votes (e.g., k=4 with 2-2 split is ambiguous).
- Common starting heuristic: $k = \sqrt{n}$, then refine via cross-validation.

The optimal k is typically found by plotting training and validation/cross-validation error against a range of k values and selecting the k at the minimum validation error (the classic U-shaped curve).

3.3 Distance Metrics

The choice of distance metric defines what "nearest" means, and dramatically affects results:

Metric	Formula	Best For
Euclidean (L2)	$\sqrt{\sum (x_i - y_i)^2}$	Continuous numeric, default choice
Manhattan (L1)	$\sum x_i - y_i $	High-dimensional, robust to outliers
Minkowski	$(\sum x_i - y_i ^p)^{1/p}$	Generalised form (p=1 Manhattan, p=2 Euclidean)
Cosine Distance	$1 - (x \cdot y) / (x * y)$	Text, sparse high-dimensional vectors
Hamming Distance	count of mismatched positions	Categorical/binary attributes
Mahalanobis	$\sqrt{(x - y)^T S^{-1} (x - y)}$	Correlated features, scale-invariant

3.4 Aspects to Consider When Designing kNN

Figure 3.2 – kNN Design Considerations Map

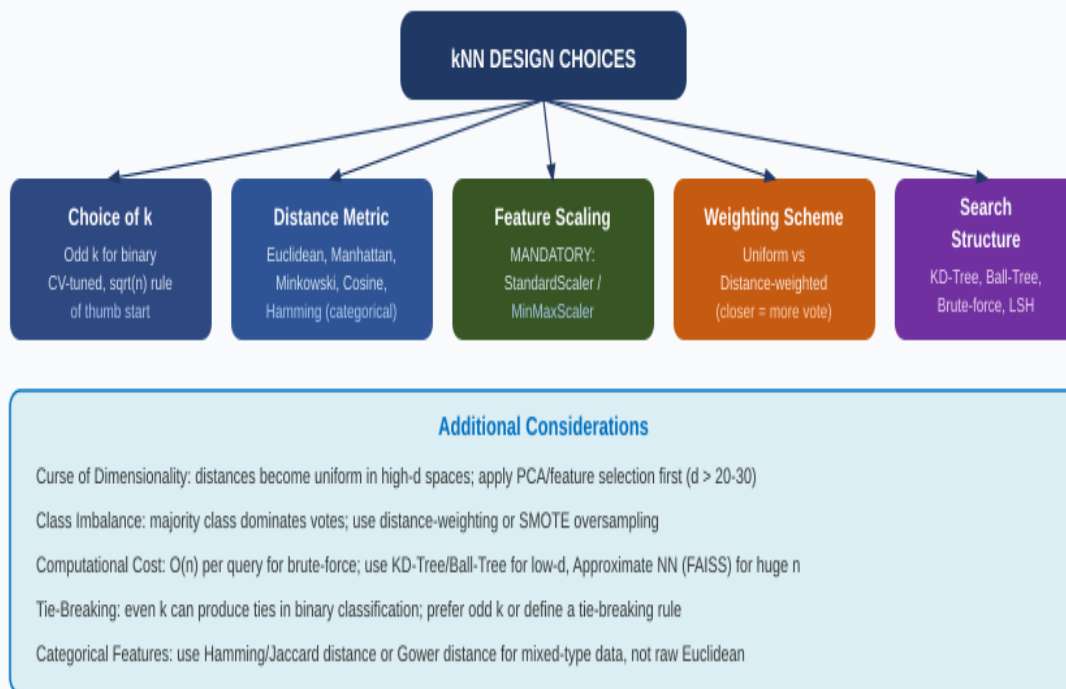


Figure 3.2 – kNN Design Considerations Map

3.4.1 Feature Scaling (Mandatory)

Because kNN relies entirely on distance, features with larger numeric ranges dominate the distance calculation unless all features are scaled to comparable ranges. StandardScaler (Z-score) or MinMaxScaler must always be applied before kNN — this is non-negotiable, unlike for tree-based or NB models.

3.4.2 Weighting Scheme

Uniform weighting: every one of the k neighbours contributes an equal vote. Distance weighting: closer neighbours contribute more strongly to the vote (commonly $\text{weight} = 1/\text{distance}$), reducing the influence of neighbours that are technically "in the k" but actually far away. sklearn: `KNeighborsClassifier(weights="distance")`.

3.4.3 Handling Categorical and Mixed-Type Features

Raw Euclidean distance is inappropriate for categorical attributes. Options: one-hot encode nominal features (increases dimensionality), use Hamming distance for purely categorical data, or use Gower distance to combine numeric and categorical attributes in a single unified metric (see Unit II, Topic 3).

3.4.4 The Curse of Dimensionality

As the number of features grows large (typically beyond 20-30 dimensions), all pairwise distances tend to converge toward similar values, making the concept of "nearest" neighbour increasingly meaningless. Mitigation: apply PCA or feature selection to reduce dimensionality before applying kNN.

3.4.5 Class Imbalance

In imbalanced datasets, the majority class tends to dominate the k-nearest-neighbour vote even near minority-class regions. Mitigation: distance-weighted voting, SMOTE oversampling of the minority class, or adjusting the effective k per class.

3.4.6 Computational Efficiency and Search Structures

Naive brute-force search computes distance to every training point: $O(n)$ per query, $O(n)$ memory. For large n , efficient spatial data structures avoid scanning the full dataset:

- KD-Tree: efficient for low-dimensional data ($d < \sim 20$), partitions space along axes. sklearn: `algorithm="kd_tree"`.
- Ball-Tree: better than KD-Tree for higher dimensions, partitions space into nested hyperspheres. sklearn: `algorithm="ball_tree"`.
- Approximate Nearest Neighbor (ANN): for very large n (millions+), libraries like FAISS or Annoy trade a small amount of accuracy for massive speed gains via approximate search.

3.4.7 Tie-Breaking

When k is even (or more generally when votes tie), a deterministic tie-breaking rule is needed: fall back to the nearest single neighbour, reduce k by 1, or use distance-weighted voting which rarely produces exact ties.

3.5 Selecting Optimal k via Cross-Validation

The standard procedure: for a range of candidate k values, perform k -fold cross-validation, compute the average validation error for each k , and select the k that minimises this error. This typically produces a U-shaped error curve.

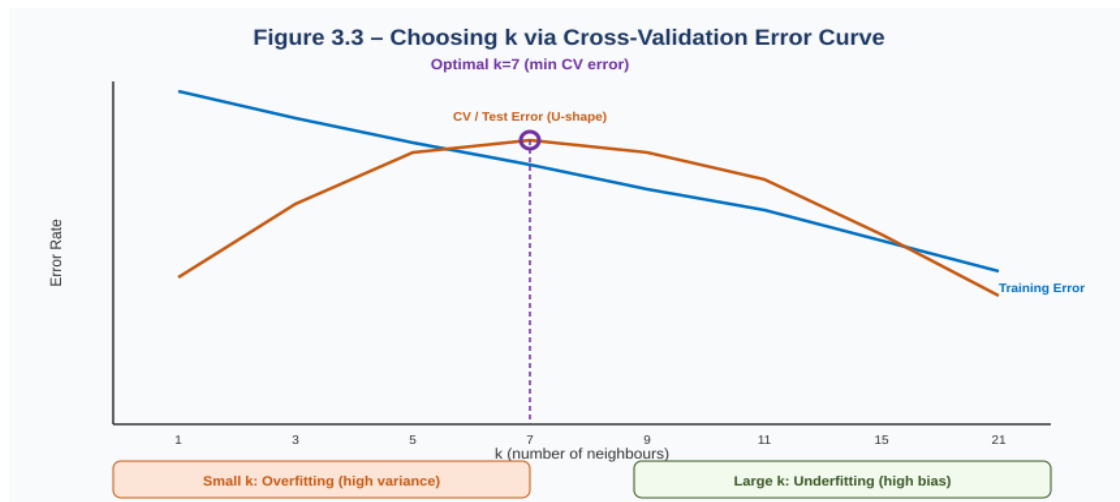


Figure 3.3 – Choosing k via Cross-Validation Error Curve

4. Architecture / kNN Prediction Workflow

Step	Action	Method / Code	Output
1	Scale features	<code>StandardScaler().fit_transform(X_train)</code>	Standardised feature matrix
2	Choose distance metric	<code>metric="euclidean"/"manhattan"/"cosine"</code>	Distance function for comparisons
3	Tune k via cross-validation	<code>GridSearchCV</code> over $k=1..30$	Optimal k value

4	Choose weighting scheme	weights="uniform" or "distance"	Voting strategy
5	Build search structure	algorithm="kd_tree"/"ball_tree"/"brute"	Efficient index for queries
6	Predict for query point	model.predict(X_query)	Predicted class label
7	Validate on held-out test set	accuracy_score, confusion_matrix	Final performance metrics

5. Industry Tools and Technologies

5.1 Commercial Tools

Tool	Vendor	kNN Features
SPSS Modeler	IBM	kNN node with auto-k selection and distance metric options
SAS Enterprise Miner	SAS	PROC DISCRIM with nonparametric kNN method
RapidMiner	RapidMiner	k-NN operator with weighted vote and mixed-measure options
Azure ML Studio	Microsoft	Multiclass/Two-Class kNN module
Elasticsearch	Elastic	Approximate kNN search for vector similarity at scale

5.2 Open-Source Tools

- scikit-learn: KNeighborsClassifier, KNeighborsRegressor, NearestNeighbors
- FAISS (Facebook AI): high-performance approximate nearest neighbour search for billions of vectors
- Annoy (Spotify): approximate nearest neighbours optimised for memory-mapped large-scale search
- SciPy: scipy.spatial.cKDTree, scipy.spatial.distance for custom distance computations
- class (R): knn() function from the class package

6. Real-World Applications

Domain	Application	Key Design Choice	Outcome
Recommender Systems	User-based collaborative filtering	Cosine similarity, weighted voting	Personalised product recommendations
Healthcare	Patient similarity for treatment recommendation	Gower distance for mixed-type records	Identify similar past cases
Computer Vision	Image retrieval / face recognition (after embedding)	Approximate NN (FAISS) on embeddings	Sub-second search over millions of images
Finance	Credit risk scoring via similar applicant profiles	Standardised Euclidean distance	Risk tier assignment

Anomaly Detection	Network intrusion detection	Distance to k-th nearest neighbour as anomaly score	Flags outlying connection patterns
-------------------	-----------------------------	---	------------------------------------

7. Advantages and Limitations

Advantages	Limitations
Simple, intuitive, no training phase required	Slow prediction for large datasets without indexing ($O(n)$ brute-force)
Naturally handles multi-class problems	Highly sensitive to feature scaling — requires careful preprocessing
No assumptions about underlying data distribution	Performance degrades sharply with high dimensionality (curse of dimensionality)
Can model highly irregular, non-linear decision boundaries	Memory-intensive — must store the entire training set
Easy to update incrementally (just add new points)	Sensitive to irrelevant/redundant features and class imbalance

8. Case Study

CASE STUDY

Case: MNIST Handwritten Digit Recognition with kNN

Problem: A classic benchmark task — classify 28x28 pixel grayscale images of handwritten digits (0-9) into the correct digit class.

Dataset: MNIST: 60,000 training images, 10,000 test images, 784 raw pixel features per image.

Baseline Attempt: Applying kNN directly on raw 784-dimensional pixel vectors with Euclidean distance and $k=5$: test accuracy = 96.9%, but prediction time per query = 1.2 seconds using brute-force search over 60,000 training images — far too slow for real-time use.

Optimisation 1 — Dimensionality Reduction: Applied PCA reducing 784 dimensions to 50 principal components (95% variance retained). Distance computations became 15x faster with negligible accuracy change (96.7%).

Optimisation 2 — Search Structure: Replaced brute-force search with a KD-Tree (suitable since reduced dimensionality is now only 50). Query time dropped to 0.04 seconds — a 30x speed-up.

Optimisation 3 — k Tuning: Cross-validation over $k=1$ to 15 found $k=3$ optimal for this PCA-reduced feature space, slightly improving accuracy to 97.2%.

Conclusion: Through PCA dimensionality reduction, an appropriate search structure, and systematic k-tuning, kNN achieved 97.2% accuracy with sub-50ms query latency — demonstrating that careful design (not algorithm complexity) often determines whether kNN is production-viable.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: Unscaled Features Cause kNN to Ignore Most Predictive Variable

Background: A real-estate startup builds a kNN model to find "similar homes" using features:

square_footage (500-5000), num_bedrooms (1-6), distance_to_city_center_km (0-50), year_built (1900-2024).

Technical Environment: Python, scikit-learn KNeighborsRegressor, 40,000 property records.

Problem Statement: The model consistently recommends homes built in similar years as "most similar," largely ignoring square footage and bedroom count, which intuitively should matter more to buyers.

Available Evidence: year_built ranges from 1900-2024 (range=124), while num_bedrooms ranges 1-6 (range=5). Raw Euclidean distance is dominated by year_built's much larger numeric range, even though it is not necessarily the most important feature.

Investigation: Compute per-feature contribution to total squared distance across a sample of query-neighbour pairs — confirm year_built contributes ~85% of total squared distance on average.

Analysis: Apply StandardScaler to standardise all features to mean=0, std=1 before computing distances.

Findings: After scaling, similar-home recommendations shifted to be driven by square footage and bedroom count as the dominant factors, matching real-estate agent intuition. User satisfaction surveys on recommendation relevance improved from 61% to 88%.

Recommended Solution: Always apply StandardScaler (or MinMaxScaler) to all numeric features before fitting kNN — this is not optional. Validate post-scaling that no single feature dominates distance calculations using per-feature distance contribution analysis.

SCENARIO 2

SCENARIO 2: Curse of Dimensionality in Gene Expression Classification

Background: A bioinformatics team applies kNN directly to a gene expression dataset with 20,000 gene features per patient sample to classify cancer subtype (4 classes), using only 300 patient samples.

Technical Environment: Python, scikit-learn KNeighborsClassifier, StandardScaler applied.

Problem Statement: Despite proper scaling, kNN test accuracy is only 41% (barely above random for 4 classes at 25%), far below the 88% achieved by a Random Forest on the same data.

Available Evidence: With $d=20,000$ and $n=300$, this is a severe high-dimensional, small-sample scenario. Pairwise distance distribution analysis shows that the ratio of max-to-min distance among all sample pairs is only 1.08 — nearly all points are approximately equidistant, exactly the symptom of the curse of dimensionality.

Investigation: Apply PCA to reduce dimensionality before kNN, retaining components that explain 95% of variance.

Analysis: PCA reduces 20,000 genes to 42 principal components capturing 95% variance. Re-compute pairwise distances: max-to-min ratio improves to 3.4, indicating distances are now meaningfully discriminative.

Findings: kNN on 42 PCA components: accuracy improves from 41% to 79%. Tuning k (optimal $k=5$) and switching to distance-weighted voting pushed accuracy to 83% — now competitive with Random Forest.

Recommended Solution: Never apply kNN directly to very high-dimensional data ($d \gg n$ or $d > \sim 30-50$) without dimensionality reduction first. Always sanity-check the distance distribution (max/min ratio) before trusting kNN results in high-dimensional settings.

SCENARIO 3

SCENARIO 3: Class Imbalance Causes kNN to Always Predict Majority Class

Background: A bank builds a kNN model for fraud detection: 99.7% legitimate transactions, 0.3% fraudulent, using transaction amount, time, location, and merchant category features.

Technical Environment: Python, scikit-learn KNeighborsClassifier($k=15$), StandardScaler, 500,000 transaction records.

Problem Statement: The model achieves 99.6% accuracy but a fraud recall of only 8% — it almost never flags actual fraud, since the test set is dominated by legitimate transactions.

Available Evidence: With $k=15$ and a 99.7%/0.3% class ratio, the expected number of fraud neighbours among any 15 nearest neighbours (even directly adjacent to genuine fraud clusters) is often fewer than the legitimate-class count nearby, due to the overwhelming base rate of legitimate transactions throughout the feature space.

Investigation: Examine vote composition for known fraud cases in the test set — confirm that even when several true neighbours are fraudulent, legitimate transactions still frequently outnumber them within $k=15$.

Analysis: Reduce k (fewer neighbours means less dilution by the majority class), switch to distance-weighted voting (closer neighbours matter more, reducing the influence of distant majority-class points), and apply SMOTE oversampling on the minority (fraud) class in the training set.

Findings: Combining $k=5$, distance-weighted voting, and SMOTE oversampling: fraud recall improved to 71%, with precision at 64% (an acceptable trade-off given the cost asymmetry of missed fraud vs false alarms).

Recommended Solution: For imbalanced classification with kNN, reduce k , use distance weighting, and apply oversampling (SMOTE) or undersampling techniques on the training set — vanilla majority-vote kNN is heavily biased toward the majority class.

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Skipping feature scaling	Assuming kNN works like tree-based models	High-range features dominate distance computation	Always apply StandardScaler/MinMaxScaler before kNN
Using even k for binary classification	Not considering tie scenarios	Ambiguous predictions on ties	Use odd k , or apply distance-weighted voting
Applying kNN to very high-d data directly	Ignoring curse of dimensionality	Distances become meaningless, poor accuracy	Apply PCA/feature selection before kNN when d is large
Brute-force search on huge datasets	Not using spatial indexing structures	Extremely slow prediction (seconds per query)	Use KD-Tree/Ball-Tree (low- d) or FAISS/Annoy (large n)
Vanilla kNN on imbalanced classes	Default uniform voting with skewed class ratio	Majority class dominates predictions, poor minority recall	Reduce k , use distance weighting, apply SMOTE

11. Key Terminologies

Term	Definition
Lazy Learning	Learning paradigm with no explicit training phase; all computation deferred to prediction time
Instance-Based Learning	Classification based directly on stored training instances rather than a learned model
k (number of neighbours)	Hyperparameter controlling how many nearest training points vote on the prediction
Majority Vote	Classification rule: assign the class most common among the k nearest neighbours
Distance-Weighted Voting	Voting scheme where closer neighbours contribute more strongly than farther ones
Curse of Dimensionality	Phenomenon where distances lose discriminative power as feature dimensionality increases

KD-Tree	Spatial data structure partitioning points along axes for efficient low-dimensional NN search
Ball-Tree	Spatial data structure using nested hyperspheres, effective for moderate-to-high dimensions
Approximate Nearest Neighbor (ANN)	Fast, approximate NN search trading some accuracy for massive speed at large scale
Bias-Variance Trade-off	The balance between underfitting (high bias) and overfitting (high variance) controlled by k
Hamming Distance	Distance metric counting mismatched positions, used for categorical/binary attributes
Tie-Breaking Rule	Deterministic procedure to resolve equal-vote outcomes among classes

12. Topic Summary

UNIT III – Topic 3

K-NEAREST NEIGHBOR ALGORITHM AND DESIGN — SUMMARY

- kNN is a lazy, instance-based, non-parametric classifier: predicts via majority vote among the k closest training points.
- k controls bias-variance trade-off: small k overfits (high variance), large k underfits (high bias). Tune via cross-validation (U-shaped error curve).
- Distance metrics: Euclidean (default numeric), Manhattan (robust), Cosine (text/sparse), Hamming (categorical), Mahalanobis (correlated features).
- Feature scaling is MANDATORY for kNN — unlike tree-based or NB models, kNN is acutely sensitive to feature scale.
- Weighting: uniform (equal vote) vs distance-weighted (closer neighbours matter more) — distance weighting helps with noise and imbalance.
- Curse of dimensionality: apply PCA/feature selection before kNN when $d > \sim 20-30$, as distances become uniform and meaningless otherwise.
- Class imbalance: reduce k , use distance weighting, and/or apply SMOTE to avoid majority-class domination of votes.
- Search structures: brute-force (small n), KD-Tree/Ball-Tree (moderate n , low- d), FAISS/Annoy (large-scale approximate search).
- Tools: scikit-learn (KNeighborsClassifier/Regressor), FAISS, Annoy, scipy.spatial.